

SDP



GitHub Code Quality

Turning Maintainability Into a Merge Gate

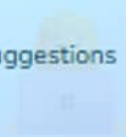
Coverage, maintainability, and reliability become a status check your merge depends on — and a predictable line item.

"How does GitHub Code Quality turn code metrics into an enforceable merge gate?"

Quality dashboards tell you what happened. Rulesets **determine what gets to merge.**

Developer

See coverage deltas and Autofix suggestions before your PR merges



Repo Admin

Wire coverage and quality thresholds into rulesets — no custom pipeline



Platform Team

Govern enablement, predict per-committer cost, and roll out safely



GA: July 20, 2026

Every team from the preview now has a real billing decision to make

~ $\frac{2}{3}$ resolved before merge

GitHub-reported finding resolution rate for Code Quality findings

\$10/committer/month

Org-wide active-committer billing — the number driving every rollout decision

PART 1

Coverage-Aware Quality Gates

From Cobertura upload to a failing status check

Build the quality gate the merge button actually checks

PART 2

Copilot Autofix in the PR Loop

Suggestions in the PR, not a follow-up ticket

Understand Autofix and the Copilot review boundary

PART 3

Reading the Bill

\$10/commmitter/month and what the rest adds up to

Model the cost before you flip the org-wide switch

PART 4

Rolling Out Without Surprises

Enable → Evaluate → Active in repeatable phases

Leave with a concrete rollout checklist for this week

Click any section to jump directly there

Coverage-Aware Quality Gates

Cobertura upload + evaluate/active ruleset = coverage as a real merge condition



Coverage Delta on Every PR

Cobertura XML upload shows the coverage delta on each PR



Ruleset-Enforced Threshold

80% minimum becomes a status check
the PR can't bypass



Evaluate Before Blocking

See what would fail without blocking
anyone yet

PR drops 4% below threshold in active mode
↳ `code-quality/coverage` → fail

Coverage Stops Being a Dashboard Number

Before Code Quality

Coverage dashboard checked weekly at best

Regressions found in incident postmortems

No PR-level signal

Coverage is a repo-level badge, not a PR delta

Coverage target is a team norm, not a gate

After Code Quality

Coverage delta shown inline on every PR

Regression caught before merge

4% drop fails the status check

Threshold enforced by a ruleset — not honor system

Copilot Autofix suggests a fix inside the PR

Per PR

coverage delta reported

Pre-merge

regression caught

0 sprints

to discover in postmortem



The same 80% target that lived in a team doc becomes a status check the merge button reads.

Feeding the Gate: Coverage Upload in CI

quality-gate-workflow.yml

```
name: CI
on:
  pull_request:
permissions:
  contents: read
  code-quality: write
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest --cov=. --cov-report=xml
      - uses: actions/upload-code-coverage@v1
        with:
          file: cobertura.xml
          language: Python
```

🔑 Required Permission

code-quality: write lets the upload step attach coverage to the PR

📄 Any Cobertura Framework

pytest-cov, JaCoCo, nyc/Istanbul — any tool that emits Cobertura XML

🏷️ Per-Service Labels

label field lets monorepos upload separate coverage reports per service

Two Enforcement Modes — One Field to Flip



Evaluate Mode

Reports without blocking

Historical PRs flagged — no merges stopped

Safe default for any new ruleset

Tune before committing

Adjust threshold until false-positive rate is acceptable

Switch to active with one field change



Active Enforcement

Blocks merges on failure

Status check required by branch protection ruleset

Exact violation shown to the PR author

Instant rollback available

Switch back to evaluate with a single field change

One threshold, one rule, one check

Two Paths to the First Coverage Upload



Agent-Generated PR

Optional onboarding path

Agent opens a reviewable PR with a least-privilege workflow

Standard upload steps, minimal permissions

Review and merge like any other PR

Available on github.com in public preview



Manual Workflow

Full control over build steps and test setup

Custom test matrices

Multi-language, monorepo, or non-standard coverage tooling

Always available

Manual authoring is never blocked by the agent path

Bring your existing CI coverage pipeline

Copilot Autofix in the PR Loop

Autofix proposes the fix; humans approve it — Code Quality does not auto-enable Copilot review



What Gets Found

CodeQL plus AI-assisted detection on every PR diff



One-Click Fix Suggestion

Developer applies, human approves — nothing auto-merges



Default-Branch Backlog

Scored backlog from default-branch scans, assignable to the agent

Code Quality finding in a PR

↳ Autofix proposed → developer applies → human approves

Finding Disposition: Without vs With Autofix

Autofix closes the loop inside the PR instead of opening a ticket

✗ Without Code Quality

- 1 PR opens with issues
Reviewer manually spots maintainability problem
- 2 Review comment added
Author adds it to the follow-up ticket backlog
- 3 PR merges anyway
Issue tracked in a ticket, not fixed
- 4 Tech-debt ticket lingers
Often never addressed



Finding becomes a ticket — often never resolved

✓ With Code Quality Autofix

- 1 PR opens
Code Quality scan posts finding with Autofix suggestion
- 2 Developer reviews suggestion
One click to apply the proposed fix
- 3 Human reviewer approves
Fix reviewed and approved within the PR
- 4 Finding resolved before merge
~ $\frac{2}{3}$ of findings resolved pre-merge



Finding resolved inside the PR

~ $\frac{2}{3}$ of findings resolved before merge with Code Quality

Two Independent Features, One PR Surface

Code Quality Autofix

Bundled with Code Quality

Included with enablement — no separate setup

Billed under Code Quality per-committer charge

AI credits consumed

AI-assisted detection and Autofix generation only

Proposals only — human approval required

Copilot Code Review

Independent enablement

Requires its own repository or organization ruleset

Not included in Code Quality

GitHub disabled auto-review in generated rulesets

Billed to the Copilot plan, not Code Quality

Enable separately if and when your team is ready

Reading the Bill

Three line items, one active-committer count, and the worked example that makes the number real



Active Committer Fee

\$10/month per active committer across the org



AI Credits

Usage-based charge for AI detection and Autofix



CodeQL Compute

Actions minutes for scan workflows

50 committers across 12 repos
↳ ~\$500+/month base before AI credits

Three Independent Line Items



Active Committers

\$10/month per person who pushed to any enabled repo in the org in the trailing 90 days

Counted once org-wide

Not per-repo

90-day rolling window



AI Credits

~\$0.01/credit for AI-assisted detection and Copilot Autofix generation runs

Scales with findings volume

Not with repo count

Separate from committer fee



CodeQL Compute

GitHub Actions minutes consumed by scan workflows, or self-hosted runner cost

Uses existing Actions budget

Runs on PR and push

Default-branch scans too

The Number This Audience Will Repeat

50 committers, 12 repos, one org

\$500+

per month base license cost for 50 active committers

50 committers × \$10/month — before AI credits or Actions minutes

🏢 Org-wide, not per-repo

Same 50 people across 12 repos = same bill as enabling on 1 repo

⚡ AI credits on top

Scales with how many findings trigger AI analysis and Autofix — not with repo count

🔍 Audit before enabling

Disabling a repo removes its committers from the active count going forward

🤖 Copilot review billed separately

Code review is a Copilot plan charge — it does not appear on the Code Quality bill

💡 Auditing which repos are enabled before the billing period starts is the single highest-leverage cost control.

How Active Committers Are Counted



90-Day Rolling Window

Anyone who pushed a commit in the trailing 90 days counts, regardless of whether they push today



Org-Wide, Not Per-Repo

The same developer across 10 repos counts once — enabling more repos does not multiply the charge



Disable to Remove Committers

Disabling Code Quality on a repo removes those committers from the active count next billing cycle



Copilot Review Not Included

Code review needs its own Copilot plan license — independent of Code Quality committer billing

Code Quality and Copilot Review Are Billed Separately

📋 Code Quality Bill

\$10/active committer/month

Org-wide, 90-day trailing window

AI credits (~\$0.01/credit)

AI-assisted detection and Autofix generation

CodeQL compute

Actions minutes for scan runs

No Copilot review charge here

🤖 Copilot Plan Bill

Copilot code review

Billed to the Copilot subscription, not Code Quality

Enabled independently

Repository or organization ruleset required

Not auto-enabled when Code Quality turns on

A separate decision your team makes when ready

Rolling Out Without Surprises

Three phases from enablement to active enforcement — leave with a concrete rollout plan for this week



Enable & Observe

Enable on pilot repos; watch scores, no enforcement yet



Evaluate Mode

Rulesets in evaluate — flag without blocking anything



Active Enforcement

Flip to active once the false-positive rate is acceptable

Rollout pattern across the preview cohort

↳ Enable → Evaluate → Active, repo group by repo group

The Evaluate-First Rollout Pattern

Each phase builds confidence before the next one applies pressure

Phase 1

Enable & Observe

Enable on pilot repos, add coverage workflow, watch quality scores — no enforcing rulesets



Phase 2

Evaluate Mode

Create rulesets with enforcement: evaluate — see what would have been blocked, tune thresholds



TARGET

Phase 3

Active Enforcement

Switch to active for pilot repos, expand to remaining repo groups — never org-wide on day one



Never skip evaluate mode — running it for at least one sprint prevents surprise merge blocks

Three Governance Layers for Deliberate Enablement



Enterprise Policy

Enterprise owners set whether Code Quality is allowed before any org can enable it

Allow for all orgs

Allow for selected orgs

Block repo-level override



Org-Level Enablement

Org admins enable Code Quality per repo, within the enterprise policy

Choose pilot repos first

Add coverage workflows

Set up evaluate rulesets



Repo-Level Control

Repo admins enable or disable within what the org allows — the billing knob for each repo

Enable: starts committer counting

Disable: removes from active count

Controlled independently

From Periodic Audit to Merge-Time Gate

Before

Coverage checked in a weekly dashboard

Quality regressions found in postmortems

No PR-level enforcement — team norms only

Org-wide rollout risks unknown blocking rate

After

Coverage delta visible on every pull request

Regressions caught before they merge

Rulesets enforce thresholds at merge time

Evaluate mode shows blast radius before going active

Per PR

coverage delta reported

Pre-merge

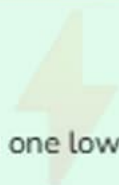
regression caught, not post-incident

Evaluate first

safe rollout path, every time

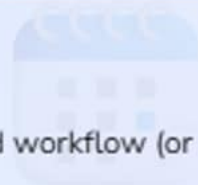
✓ What You Can Do Today

Today



- Enable Code Quality on one low-risk pilot repo
- Check if your CI produces Cobertura XML today
- Review which repos should count toward billing

This Week



- Add coverage upload workflow (or accept agent PR)
- Create a ruleset in evaluate mode at 80% coverage
- Run one full sprint in evaluate, log blocked PRs

This Month



- Flip pilot ruleset to active enforcement
- Expand to remaining org repos in groups
- Set up org dashboard review as a recurring cadence

🔑 Key Takeaway

Quality gates that used to require custom pipeline work are now one ruleset away.

 Official Documentation[GitHub Code Quality - Concepts](#)

Core concepts for PR and default-branch scanning

[Enabling GitHub Code Quality](#)

Step-by-step repo and org enablement

[Catch Issues Before Merge](#)

Ruleset-based merge gating walkthrough

[Code Quality Billing](#)

Active-committer pricing, AI credits, and Actions minutes

 Changelog Announcements[Code Quality No Longer Adds Copilot as a Reviewer](#)

Boundary between Code Quality and Copilot review enablement

[Automatic Coverage Enablement](#)

Agent-generated coverage workflow pull request in public preview



GitHub Code Quality

Turning Maintainability Into a Merge Gate

Coverage Gates

Cobertura upload → per-PR
delta → ruleset threshold →
merge condition

Autofix in the PR

Proposals inside the PR; Code
Quality and Copilot review are
independent

\$10/commmitter/month

Org-wide active-commmitter
billing plus AI credits and
Actions minutes

Enable → Evaluate → Active

Three phases; evaluate mode
first, every time, every repo
group

Which signal would your team enforce first — coverage threshold, maintainability score, or reliability score?